

HW6_JUNHO JANG

● Problem 1

(1) Starting Code (simulate_hw6_model1.R). It makes 'simul_data_hw6_model1.txt'

(2a) In hw6_model1.stan file, there are hyperparameters.

```
// Hyper(group)-parameters
vector[2] mu_p;
vector<lower=0>[2] sigma;
```

And "alpha", "beta" is posterior distribution of individual respectively.

Like stan code

```
generated quantities {
  //mu_alpha = Phi_approx(mu_p[1])
  //mu_beta = Phi_approx(mu_p[1]) * 5
}
```

We plus the code like below.

```
mu_alpha <- mean(pnorm(parameters$mu_p[, 1])) # Phi(mu_p[1])
mu_beta <- mean(pnorm(parameters$mu_p[, 2])) * 5 # Phi(mu_p[2]) * 5
```

```
mu_alpha ( mu : -0.82
mu_beta : -0.45 * 5
alpha_mean :
[1] 0.2647082 0.1531255 0.1785771 0.2367080 0.1195762 0.1896509 0.1767531
[8] 0.1856440 0.1919433 0.1310142 0.1846509 0.1822075 0.2285611 0.2020449
[15] 0.2322894 0.3006577 0.2743046 0.2700748 0.1687712 0.2631436 0.2061523
[22] 0.2057732 0.1386372 0.2542754 0.2088384 0.2564892 0.2673458 0.2106738
[29] 0.2700901 0.2233133
```

alpha_sd:

```
[1] 0.05878593 0.02830707 0.03689460 0.04308021 0.02975265 0.06612134
[7] 0.03478517 0.07266301 0.03549280 0.04728889 0.04706856 0.03766239
[13] 0.05991516 0.03840309 0.08008868 0.03654565 0.04423473 0.04904801
[19] 0.03652479 0.04577859 0.05025775 0.05505995 0.03717999 0.03868644
[25] 0.04499593 0.03445805 0.05555488 0.05002897 0.04275166 0.05432859
```

beta_mean :

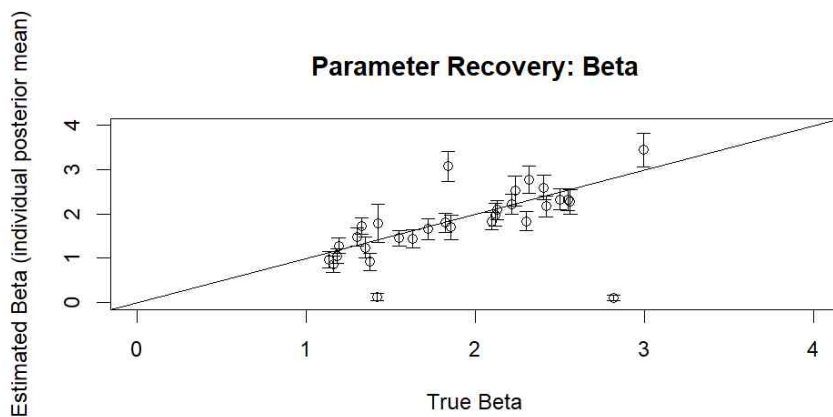
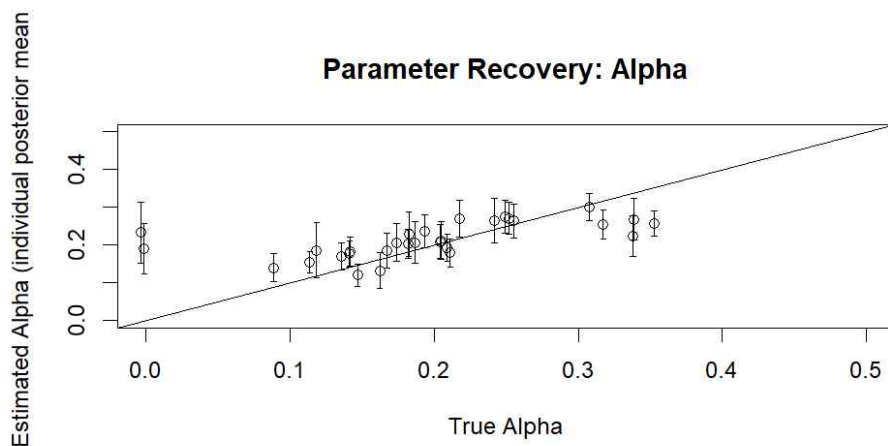
```
[1] 1.4378135 1.9794068 1.7981977 2.5188292 2.0853168 0.1000326 2.1708695
[8] 0.9146505 2.5951752 1.7834673 1.2770890 2.3190173 0.8605167 2.2179169
[15] 0.1272657 3.4472625 2.3297773 1.6530905 2.2727137 1.8315205 1.0450749
[22] 1.6980200 1.2331250 2.7755758 1.4524418 3.0765977 0.9555908 1.7205422
```

[29] 1.8320393 1.4779462

beta_sd:

```
[1] 0.20838632 0.25811575 0.22149432 0.33267018 0.21904006 0.06811107  
[7] 0.23562222 0.18919787 0.27890008 0.43707624 0.17190141 0.25250277  
[13] 0.18717391 0.21937525 0.08174219 0.37614070 0.24426525 0.24318103  
[19] 0.27758151 0.21405203 0.16540806 0.27572128 0.23625560 0.31240368  
[25] 0.17764397 0.34191269 0.18588061 0.18449602 0.19495906 0.20241391
```

(2b) There are already codes drawing plots. I only add 'title & axis name'.



Most point clustered near the diagonal ($y=x$) line. Add the error bar contains diagonal line too.

● problem 2

(1) make two codes. In stan (file name :hw6_model2_non-hierarchical.stan), we have to delete

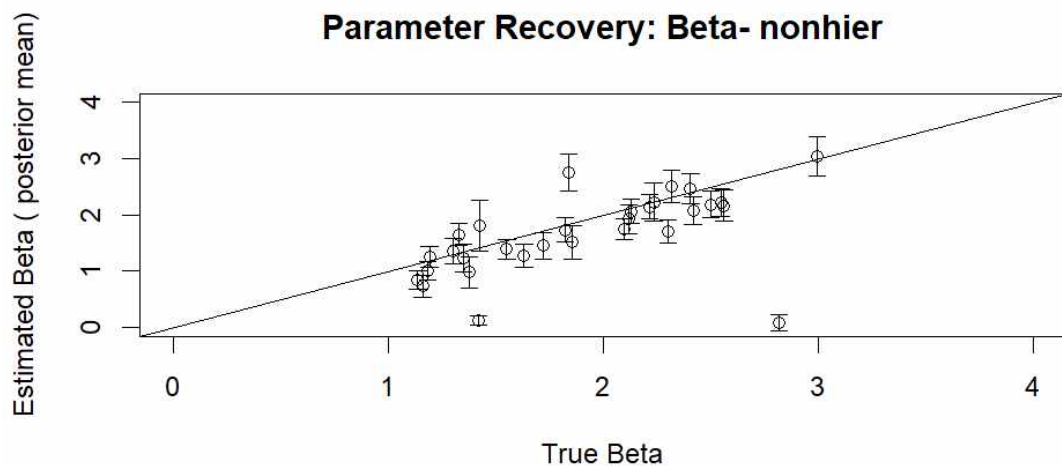
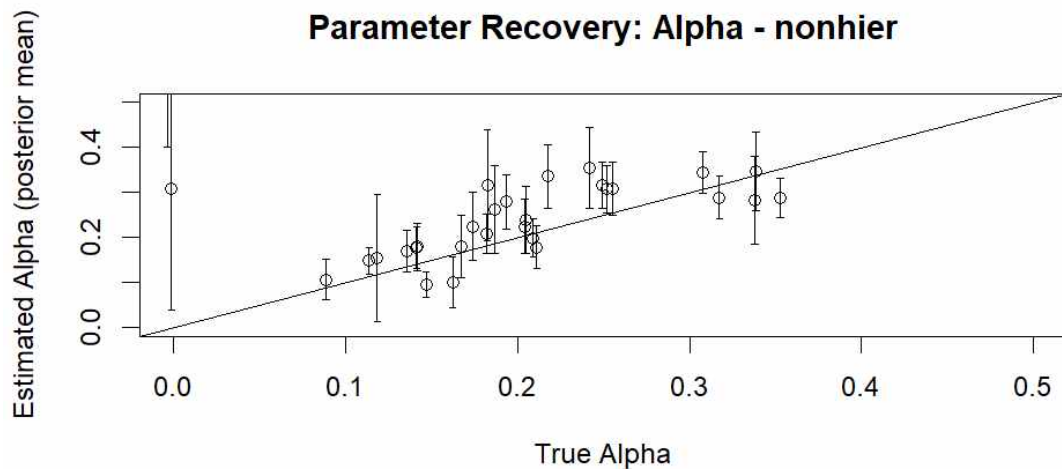
```
// Hyperparameters
//mu_p ~ normal(0, 1);
//sigma ~ normal(0, 1);
```

change the parameters

```
vector<lower=0,upper=1>[N] alpha; // learning rate [0, 1]
vector<lower=0,upper=5>[N] beta; // inverse temperature [0, 5]
```

and rename the file.

(2)



(3)

Usually non-hierarchical models have larger error bar. Also there are outlier, like

alpha[6]=0.93, alpha[15]=0.67. However we can see 'shrinkage' in herarchical model. That is because hierarchical model has group distribution. It can pull the estimated value of the subject toward the group average. But non - hierarchical models independently estimates each subject, so the posterior is very wide,

● problem 3

(1) change number of subjects and number of trials

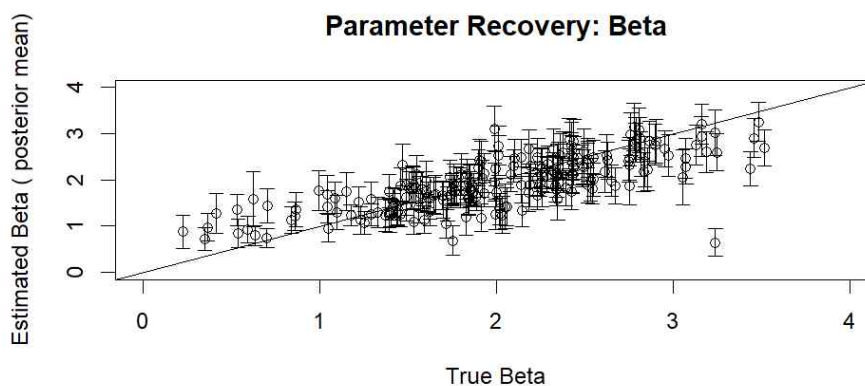
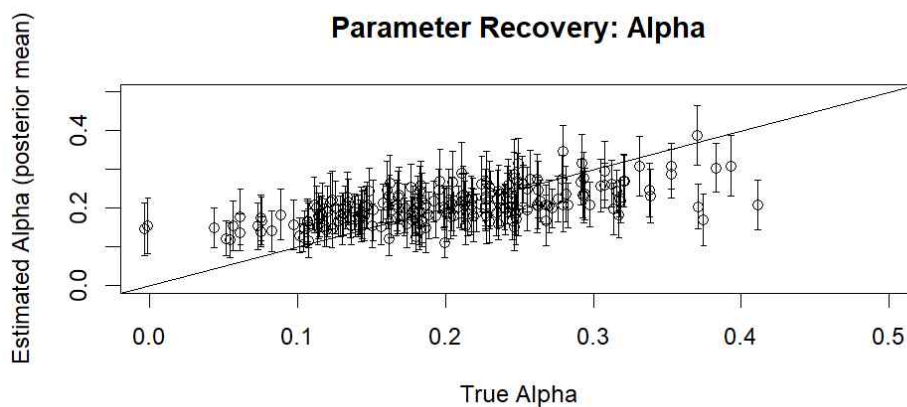
```
num_subjs <- 200 # number of subjects
```

```
num_trials <- 100 # number of trials per subject
```

(2) Run 'hw6_model1'

Because we change 'data' not 'code'.

(3)



Like Q1, dots are near to diagonal. each dots have smaller error bar. But Alpha doesn't. Most of dots are near 'alpha == 0,2' It means there is a shrinkage.

(4) Alpha is different. It is because the number of trials decline (300 to 100). Alpha means learning rate which can be acquired by amount of trials. Smaller trials make model depends on 'group mean', not individual data.

However, Beta means 'select pattern' that how individuals selects consistently. It can be obtained 100 trials.

● problem 4

(1)

mu_alpha_pos: 0.15

mu_alpha_neg: 0.308

mu_beta: 2.145

alpha_pos_mean:

[1] 0.25862873 0.19021276 0.23622526 0.13795482 0.11995698 0.11635612
[7] 0.09490096 0.14153136 0.19901534 0.10425101 0.12637499 0.21338820
[13] 0.20110990 0.10725296 0.09150963 0.22865369 0.14527892 0.32929993
[19] 0.15205275 0.23679325 0.18346016 0.20741347 0.12634367 0.10177184
[25] 0.13977135 0.22381834 0.31355833 0.11477003 0.26912092 0.15820314

alpha_pos_sd :

[1] 0.10705489 0.08894927 0.10967519 0.07921188 0.10950270 0.12728165
[7] 0.07737316 0.08589091 0.13252592 0.06964884 0.05611389 0.13888730
[13] 0.07067677 0.08524907 0.09959511 0.08122940 0.05888899 0.16193137
[19] 0.09127628 0.13550585 0.17061204 0.08134074 0.11731315 0.07231558
[25] 0.06489923 0.08687676 0.14886981 0.07883528 0.09024397 0.05159436

alpha_neg_mean :

[1] 0.3239812 0.3259121 0.2933239 0.2997569 0.2953817 0.3120340 0.3153185
[8] 0.3137252 0.3196108 0.3029099 0.3073658 0.3144714 0.3054972 0.3086891
[15] 0.3128628 0.3102441 0.3091638 0.3213028 0.3085132 0.3253333 0.2988177
[22] 0.3138774 0.3154991 0.3044184 0.3185936 0.2980709 0.3095950 0.2925846
[29] 0.3167513 0.3012566

alpha_neg_sd :

[1] 0.06495967 0.06413932 0.05955914 0.05997246 0.06479273 0.07101636
[7] 0.06942190 0.06225622 0.06415055 0.06102023 0.05754808 0.06024996
[13] 0.05449393 0.06187307 0.06825698 0.06588329 0.05823320 0.06180181
[19] 0.06382739 0.05986085 0.06114691 0.06048176 0.06794818 0.06714843
[25] 0.05937958 0.05925581 0.06358671 0.05884907 0.05523173 0.05803954

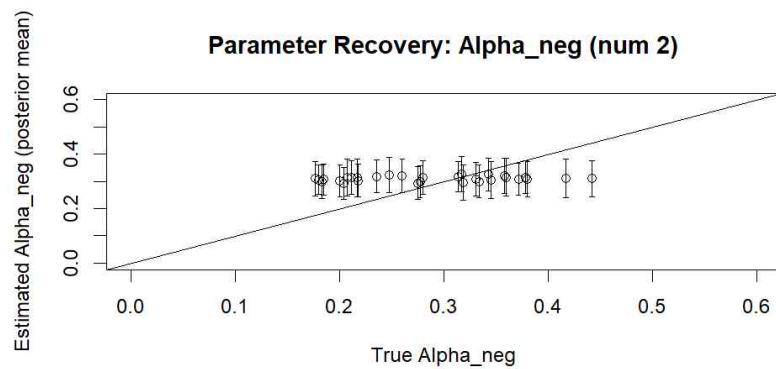
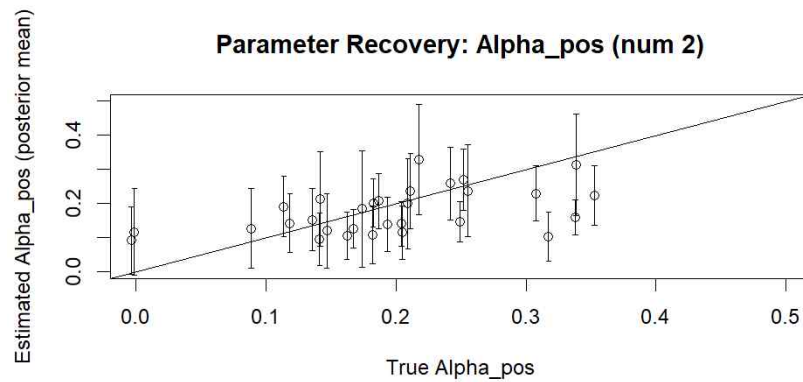
beta_mean :

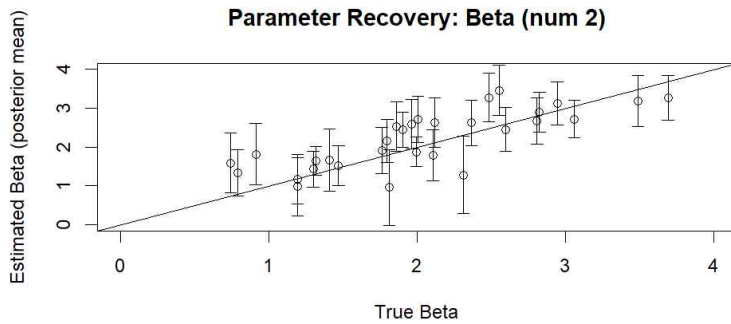
```
[1] 2.4420211 2.6245956 2.5288173 2.1649237 1.3319998 0.9578174 1.5857348
[8] 2.6714285 1.4263191 1.7816895 2.6251790 2.7088435 3.1236584 1.6661099
[15] 1.2725303 1.6432078 3.1922728 3.2761008 1.9090748 3.4583503 1.1710784
[22] 2.7158111 0.9754110 1.8166053 2.5859671 2.4529459 1.8746692 1.5167594
[29] 2.8925266 3.2644437
```

beta_sd :

```
[1] 0.4566870 0.6336002 0.6305403 0.5553664 0.5951825 0.9817272 0.7735681
[8] 0.5933623 0.4716063 0.6628436 0.5869588 0.6003753 0.5622563 0.8058848
[15] 0.9980952 0.3770066 0.6581771 0.6203126 0.6001445 0.6430156 0.6401666
[22] 0.4796305 0.7509710 0.7852898 0.6435721 0.5610713 0.3791141 0.5197776
[29] 0.5157503 0.5688436
```

(2)



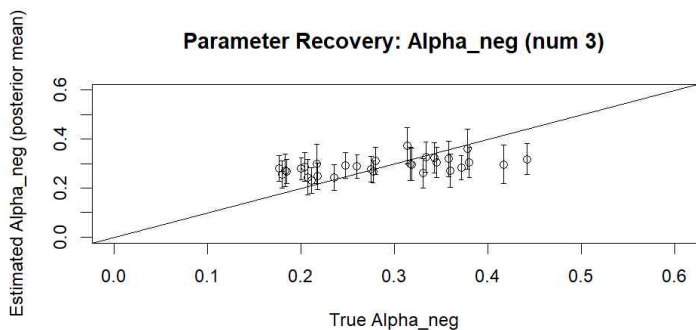
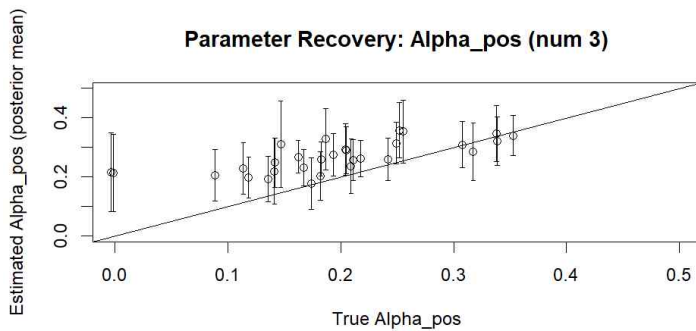


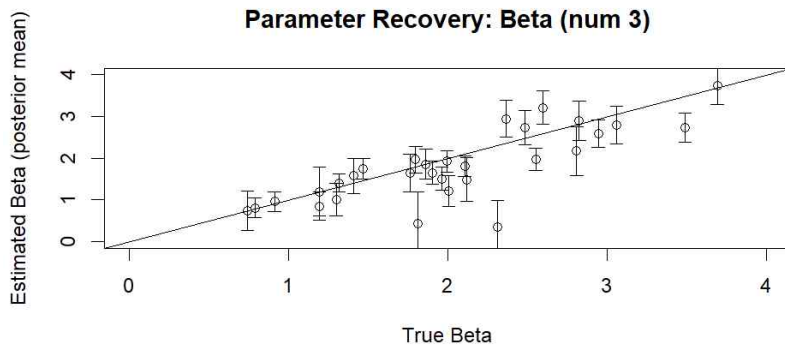
Beta was relatively well recovered. But α_{pos} and α_{neg} have wide error bar. Especially α_{neg} all subjects' estimates clustered around 0.30. It means individual differences in α_{neg} could not be distinguished from the group mean.

This pattern is likely due to the task structure. With $\text{pr_correct_option2} = 0.8$ and only 100 trials, subjects chose option 2 and received positive outcomes most of the time.

As a result, negative prediction errors ($\text{PE} < 0$) occurred infrequently. It supply insufficient information to estimate α_{neg} at the individual level. So most of data pulled toward group mean($\mu_{\alpha_{\text{neg}}}$).

(3)





(4)

Compared to Question 4-2, parameter recovery for α_{neg} improved in 4-3. There are two reasons. First, the number of trials increased (110 $\rightarrow$ 300). It provides more data. Second, the reward probabilities changed. Becoming $\text{pr_correct_option2}$ is 0.6. This means negative prediction errors ($\text{PE} < 0$) occur more often. As a result, there is substantially more information in the data to estimate α_{neg} at the individual level. So α_{neg} recovered well.

● problem 5

(1) 10 * 10 * 10

```
library(ggplot2)
library(reshape2)
data <- read.table("ra_exampleData.txt", header=TRUE)
subj1 <- subset(data, subjID == unique(data$subjID)[1])
print(subj1)
# Grid
rho_grid <- seq(0.1, 2.0, length.out = 10) # change length
lambda_grid <- seq(0.1, 10.0, length.out = 10)
tau_grid <- seq(0.1, 5.0, length.out = 10)
#param_prospect_low <- c(0, 0, 0); param_prospect_up <- c(2, 10, 5) #Use 'ra_model_select' / rho,
lambda, tau
# posterior
posterior <- array(0, dim = c(10, 10, 10)) # 10 * 10 * 10 or 30 * 30 * 30
# likelihood
log_likelihood <- function(rho, lambda, tau, ra_data) {
  T <- nrow(ra_data)
  cert <- ra_data$cert
  gain <- ra_data$gain
  loss <- ra_data$loss
  gamble <- ra_data$gamble

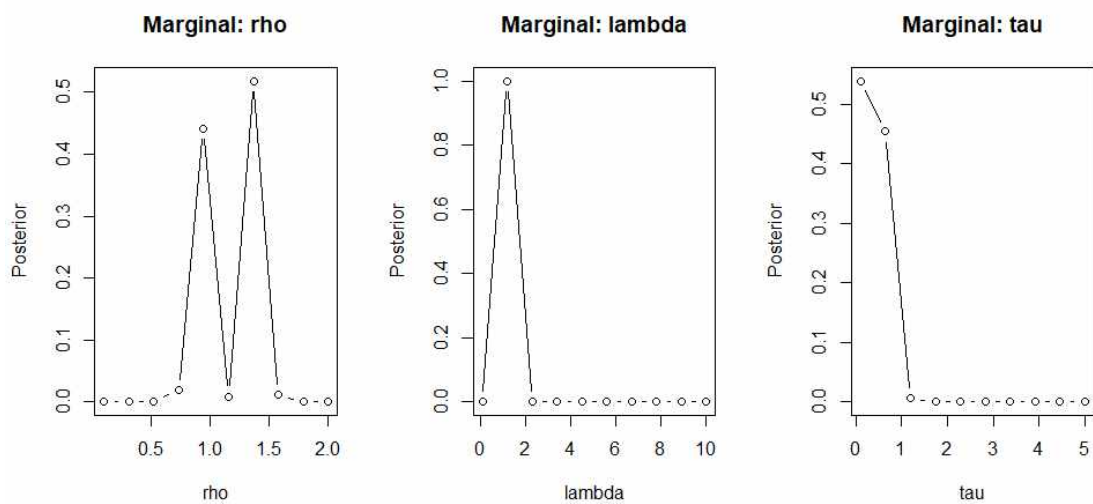
  loglik <- 0
  for (t in 1:T) {
    evSafe <- cert[t]^rho
```



```

evGamble <- 0.5 * (gain[t]^rho - lambda * abs(loss[t])^rho)
pGamble <- 1 / (1 + exp(tau * (evSafe - evGamble)))
pGamble <- pGamble * 0.9998 + 0.0001
loglik <- loglik + gamble[t]*log(pGamble) + (1-gamble[t])*log(1-pGamble)
}
return(loglik)
}
# Grid calculation
for (i in 1:10) {
  for (j in 1:10) {
    for (k in 1:10) {
      ll <- log_likelihood(rho_grid[i], lambda_grid[j], tau_grid[k], subj1)
      # uniform prior → log_prior = 0
      posterior[i, j, k] <- exp(ll)
    }
  }
}
# normalization
posterior <- posterior / sum(posterior)
# marginal posterior
post_rho <- apply(posterior, 1, sum)
post_lambda <- apply(posterior, 2, sum)
post_tau <- apply(posterior, 3, sum)
# draw plots
par(mfrow = c(1, 3))
plot(rho_grid, post_rho, type="b", xlab="rho", ylab="Posterior", main="Marginal: rho")
plot(lambda_grid, post_lambda, type="b", xlab="lambda", ylab="Posterior", main="Marginal: lambda")
plot(tau_grid, post_tau, type="b", xlab="tau", ylab="Posterior", main="Marginal: tau")
par(mfrow = c(1, 1))

```



(2)

```

library(ggplot2)
library(reshape2)

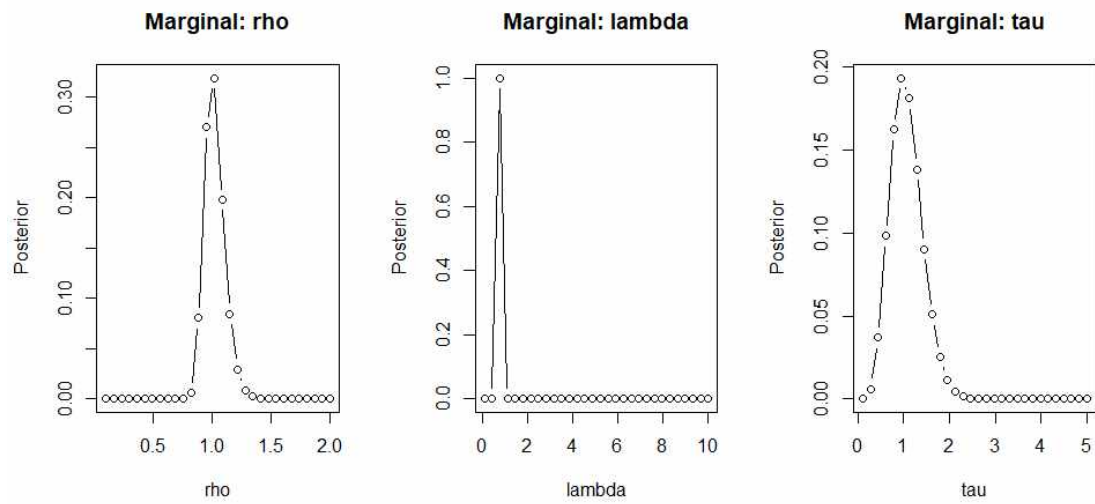
```

```

data <- read.table("ra_exampleData.txt", header=TRUE)
subj1 <- subset(data, subjID == unique(data$subjID)[1])
print(subj1)
# Grid
rho_grid <- seq(0.1, 2.0, length.out = 30) # change length
lambda_grid <- seq(0.1, 10.0, length.out = 30)
tau_grid <- seq(0.1, 5.0, length.out = 30)
#param_prospect_low <- c(0, 0, 0); param_prospect_up <- c(2, 10, 5) #Use 'ra_model_select' / rho,
lambda, tau
# posterior
posterior <- array(0, dim = c(30, 30, 30)) # 10 * 10 * 10 or 30 * 30 * 30
# likelihood
log_likelihood <- function(rho, lambda, tau, ra_data) {
  T <- nrow(ra_data)
  cert <- ra_data$cert
  gain <- ra_data$gain
  loss <- ra_data$loss
  gamble <- ra_data$gamble

  loglik <- 0
  for (t in 1:T) {
    evSafe <- cert[t]^rho
    evGamble <- 0.5 * (gain[t]^rho - lambda * abs(loss[t])^rho)
    pGamble <- 1 / (1 + exp(tau * (evSafe - evGamble)))
    pGamble <- pGamble * 0.9998 + 0.0001
    loglik <- loglik + gamble[t]*log(pGamble) + (1-gamble[t])*log(1-pGamble)
  }
  return(loglik)
}
# Grid calculation
for (i in 1:30) {
  for (j in 1:30) {
    for (k in 1:30) {
      ll <- log_likelihood(rho_grid[i], lambda_grid[j], tau_grid[k], subj1)
      # uniform prior → log_prior = 0
      posterior[i, j, k] <- exp(ll)
    }
  }
}
# normalization
posterior <- posterior / sum(posterior)
# marginal posterior
post_rho <- apply(posterior, 1, sum)
post_lambda <- apply(posterior, 2, sum)
post_tau <- apply(posterior, 3, sum)
# draw plots
par(mfrow = c(1, 3))
plot(rho_grid, post_rho, type="b", xlab="rho", ylab="Posterior", main="Marginal: rho")
plot(lambda_grid, post_lambda, type="b", xlab="lambda", ylab="Posterior", main="Marginal: lambda")
plot(tau_grid, post_tau, type="b", xlab="tau", ylab="Posterior", main="Marginal: tau")
par(mfrow = c(1, 1))

```



By changing grid num, the posterior distributions became smoother and more clearly. That is why the number of calculation increased. (1,000 $\rightarrow$ 27,000) rho showed a single peak near 1.1. Lambda showed low values. And tau revealed more smooth distribution near 0.6 ~ 1.4.